

Lecture 9: Dimensionality Reduction

Spring 2026

10 ECTS Credits

Department of Information and Communication Technology

Faculty of Engineering and Science

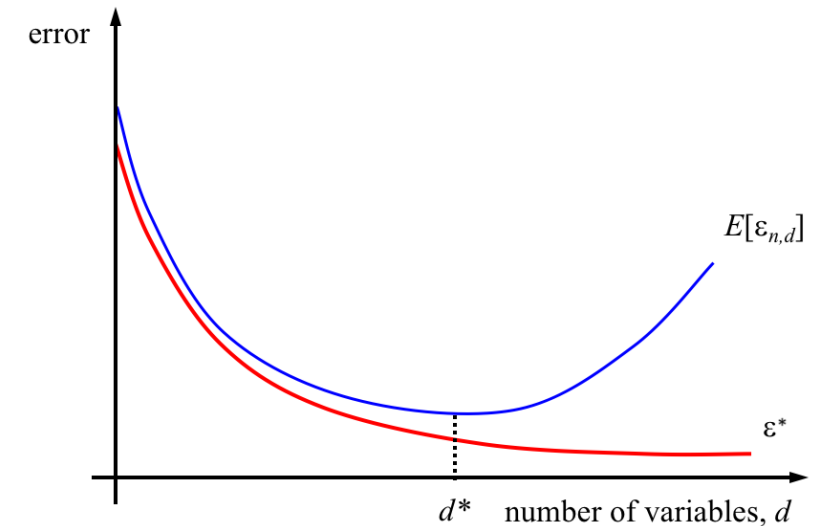
Recap

- Model Selection for Classification
- Classification Complexity:
 - Model parameters vs. Free parameters
 - Approximation Error
 - Design Error
 - Complexity Dilemma
 - Curse of Dimensionality
- Model Selection Methods:
 - Validation Error Minimization
 - Training-Validation-Testing Strategy
 - Training Error Minimization



Dimensionality Reduction

- Peaking phenomenon or the curse of dimensionality results in increase in classification error beyond a point as we add more features.
- Optimal dimensionality is dependent on the sample size, and classification rule complexity.
- Thus, in some cases dimensionality reduction is necessary.
- Other reasons:
 - Reduce computational load,
 - Improving classifier interpretability, and
 - Allowing visualization of data in low-dimensional spaces.



Feature Extraction for Classification

- Given the original vector of measurements $\mathbf{X} \in \mathbb{R}^p$, dimensionality reduction finds a transformation $T: \mathbb{R}^p \rightarrow \mathbb{R}^d$ where $d < p$, such that the new feature vector is $\mathbf{X}' = T(\mathbf{X}) \in \mathbb{R}^d$.
- This process is also called **feature extraction**, in the image processing domain.
- The transformed feature vector $\mathbf{X}'$ should be such that it maximizes a **class-separability criterion** $J(\mathbf{X}', Y)$, which allows for the minimization of loss of discriminatory information.
- Common class separability criteria:
 - The Bayes Error
 - The Mahalanobis Distance
 - Given a classification rule Ψ_n , the designed classification error
- A feature extraction transformation $\mathbf{X}' = T(\mathbf{X})$ is said to be lossless for the assumed class separability criterion, if $J(\mathbf{X}', Y) = J(\mathbf{X}, Y)$.
- Feature extraction transformation is lossless, if it is invertible.

Feature Selection

- Simplest way to reduce dimensionality of a problem.
- Discard features that are redundant or less informative about the label.
- Useful where:
 - Interpretability of the classifier is crucial, and
 - Complicated feature transformations are undesirable.
- Examples:
 - When a small subset of genes predicts a disease or condition.
 - Selecting vibration measurements in predicting failure of a machine.
- We will see some feature selection methods in the next slides.

Exhaustive Search

- Searching through the different combinations of features and select the ones that maximizes class separability criteria.
- Let $A \subset \{1, \dots, p\}$ be a subset of indices of the features $\mathbf{X} \in \mathbb{R}^p$ such that it defines a set of features $\mathbf{X}_A$ indexed by A .
- Example: $\mathbf{X}_A = \{X_1, X_p\}$ is a 2-dimensional vector containing first and last features in the original feature vector.
- Let $J(A) = J(\mathbf{X}_A, Y)$ be a class separability criterion given A .
- The exhaustive feature selection problem is to find A^* such that:
$$A^* = \arg \max_{|A|=d} J(A)$$
- The optimal solution is guaranteed to be reached in finite term by exhaustive search.

Wrapper vs. Filter Feature Selection

- **Wrapper Feature Selection:**
 - We use the classification error as the class-separability criteria in selecting the optimal feature subset.
 - The classification rule Ψ_n used in the criterion is same as the classification rule used to train the final classifier.
- **Filter Feature Selection:**
 - All other cases.
 - Example: using Mahalanobis distance as class-separability criterion.
- Wrapper feature selection can fit the data better due to matching the search to the final desired classifier.
- However, it can lead to selection bias and overfitting in small-sample cases.

Exhaustive Search Complexity

- The number of subsets to be evaluated in an exhaustive search is given by:

$$m = \binom{p}{d} = \frac{p!}{d! (p - d)!}$$

- The complexity of feature selection is thus $\mathcal{O}(p^d)$
- Suppose there are 10 original features, and we select 2:

$$m = \frac{10!}{2! (10 - 2)!} = 45$$

- Now if there are 100 original features and we want 10:

$$m = \frac{100!}{10! (100 - 10)!} = 17310309456440$$

Univariate Greedy Search

- Attempting to find good feature sets without an exhaustive search.
- Considering the features individually.
- In practice:
 - Retain features that are highly variable across all classes.
 - Retain one feature from each cluster of mutually correlated features.
 - Retain features that has strong correlation with the label.
- Notice:
 - A feature can be high in variance, be uncorrelated to other features, but still be noisy.
 - A feature can be highly correlated to label, but redundant because of correlation with other features.

Univariate Greedy Search

- Formally, a univariate class separability criterion $J(X_i, Y)$ is applied to each feature X_i and d features with largest J are selected.
- Example: absolute value of the test statistic of a two-sample t -test:

$$J(X_i, Y) = \frac{|\hat{\mu}_{i,0} - \hat{\mu}_{i,1}|}{\sqrt{\frac{\hat{\sigma}_{i,0}^2}{n_0} + \frac{\hat{\sigma}_{i,1}^2}{n_1}}}$$

- Another example: Using correlation analysis.
- Univariate criteria often work well, but when they fail, they fail spectacularly.

Univariate Greedy Search

- Let $d = 3$, suppose there is a feature-label distribution (X, Y) such that X_1, X_2, X_3 are independent features and

$$\varepsilon_n(X_1) < \varepsilon_n(X_2) < \varepsilon_n(X_3)$$

but such that,

$$\varepsilon_n(X_1, X_2) > \varepsilon_n(X_1, X_3) > \varepsilon_n(X_2, X_3)$$

- Univariate greedy search results in:
 - the best 2 individual features forming the worst 2-feature set,
 - the worst 2 individual features forming the best 2-feature set, and
 - The best 2-feature set does not contain the best individual feature.
- This is known as **Toussaint's Counter Example**.

Multivariate Greedy Search

- Evaluate multiple features at a time while avoiding exhaustive search.
- Achieved through sequential multivariate greedy search methods.
- Features are added or removed sequentially until a stopping criterion is met.
- Sequential methods:
 - Bottom-up searches: number of features increases over time,
 - Top-down searches: number of features decreases over time.

Sequential Forward Search

- Adds one feature at a time to the working feature set.
- Procedure:
 - Start with a null set $X_{(0)} = \emptyset$
 - Given current feature set $X_{(k)}$, evaluate the criterion $J(X_{(k)} \cup X_i, Y)$ for each $X_i \notin X_{(k)}$
 - The X_i^* that maximizes the criterion is added to the feature set:
$$X_{(k+1)} = X_{(k)} \cup X_i^*$$
 - Stop if $k = d$ or if no further improvement possible.
- SFS is simple and fast but once a feature is added, it can never be removed from the working feature set.

Sequential Backward Search

- Top-down version of SFS.
- Procedure:
 - Start with a null set $X_{(p)} = X$
 - Given current feature set $X_{(k)}$, evaluate the criterion $J(X_{(k)} \setminus X_i, Y)$ for each $X_i \in X_{(k)}$
 - The X_i^* that minimizes the drop in criterion $J(X_{(k)}, Y) - J(X_{(k)} \setminus X_i, Y)$ is removed from the feature set:
$$X_{(k+1)} = X_{(k)} \setminus X_i^*$$
 - Stop if $k = d$.
- Similar to SFS and once a feature is removed, it can never be added back to the working feature set.
- Feature sets of high dimensionality needs to be considered in the initial steps.

Generalized Sequential Search

- Generalized variation.
- Procedure:
 - Start with a null set $X_{(0)} = \emptyset$ or $X_{(p)} = X$
 - Given current feature set $X_{(k)}$, evaluate the criterion $J(X_{(k)} \cup Z_j, Y)$ or $J(X_{(k)} \setminus Z_j, Y)$ for each Z_j (a combination of a small number of features r).
 - The Z_j^* that maximizes the criterion $J(X_{(k)} \cup Z_j, Y)$ or minimizes the drop in criterion $J(X_{(k)}, Y) - J(X_{(k)} \setminus Z_j, Y)$ is added or removed from the feature set:

Generalized Sequential Forward Search (GSFS): $X_{(k+1)} = X_{(k)} \cup Z_j^*$

Generalized Sequential Backward Search (GSBS): $X_{(k+1)} = X_{(k)} \setminus Z_j^*$
- Stop if $k = d$ or no further improvements can be made (only for GSFS).
- More accurate than regular sequential search, at the expense of more computation.
- Still other finite-horizon problem and high dimensional feature vector evaluation in case of GSBS.

Plus- l Minus- r Search

- Approach solving finite-horizon problem of previous sequential search methods.
- Procedure:
 - Start from an initial feature set.
 - l features are added using SFS.
 - r features are removed using SBS.
 - Repeat the process until a stopping criterion is reached.
- Allows back-tracking, any feature can be added or removed from a working feature set.
- If $l > r$, this is bottom-up search, if $r > l$, top-down search.
- However, comes at the expense of even more computation.

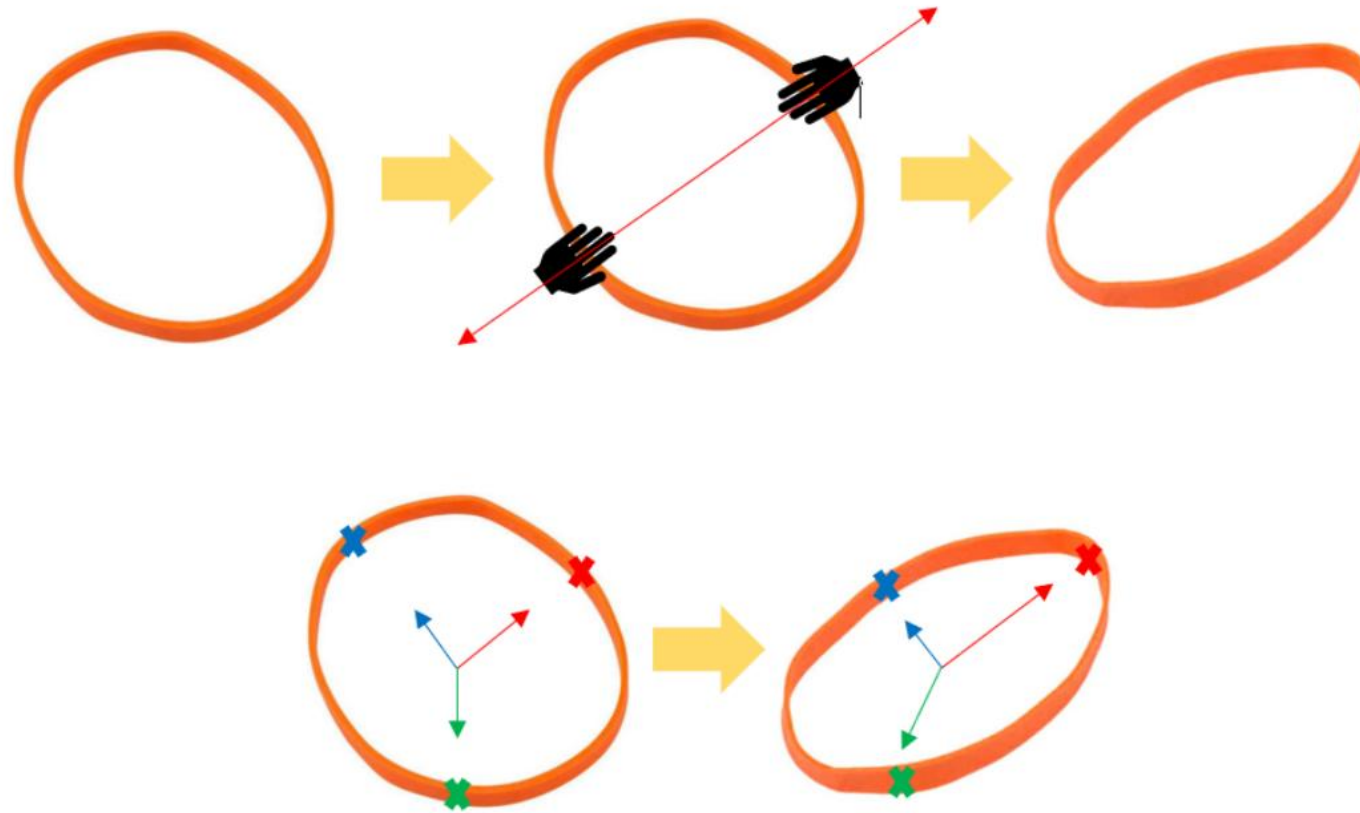
Floating search

- A variation of plus- l minus- r search, where l and r can vary at different stages of the process.
- Sequential Floating Forward Search (SFFS) is the bottom-up version, while Sequential Floating Backward Search (SFBS) is the top-down algorithm.
- Procedure (SFFS):
 - Let $X_{(0)} = \emptyset$ and find $X_{(1)}$ and $X_{(2)}$ by SFS.
 - At stage k , select feature $X_i^* \notin X_{(k)}$ that maximizes $J(X_{(k)} \cup X_i^*, Y)$. Let $Z_{(k+1)} = X_{(k)} \cup X_i^*$.
 - Find the feature $X_j^* \notin Z_{(k+1)}$ that minimizes the drop $J(Z_{(k+1)}, Y) - J(Z_{(k+1)} \setminus X_j^*, Y)$. If $X_i^* = X_j^*$, let $X_{(k+1)} = Z_{(k+1)}$, increment k and go to step 2.
 - Otherwise, start removing features from $Z_{(k)}$ to get $Z_{(k-1)}$ while $J(Z_{(k)}, Y) > J(X_{(k)}, Y)$ or until $k = 2$. Let $X_{(k)} = Z_{(k)}$, increment k and go to step 2.
 - Stop when $k = d$ or no further improvement is possible.
- Allows for optimal backtracking

Principal Component Analysis (PCA)

- Suppose you have three brothers.
- Each weigh: 74, 73, and 72 kilograms.
- You are asked to guess the brother based on their weights only.
- Can you guess accurately?
- Now suppose you are given only their heights: 147, 183, and 168 cm.
- Is it easier to guess accurately now that you have this information?
- There is more variance in their heights, making it easier to make a guess.
- Similarly, higher the variation data holds, the more discriminative information it holds.
- What if both weight and height are given?
- Similarly, what PCA should look into is the variance of different features.

Recap: Eigenvectors and Eigen values



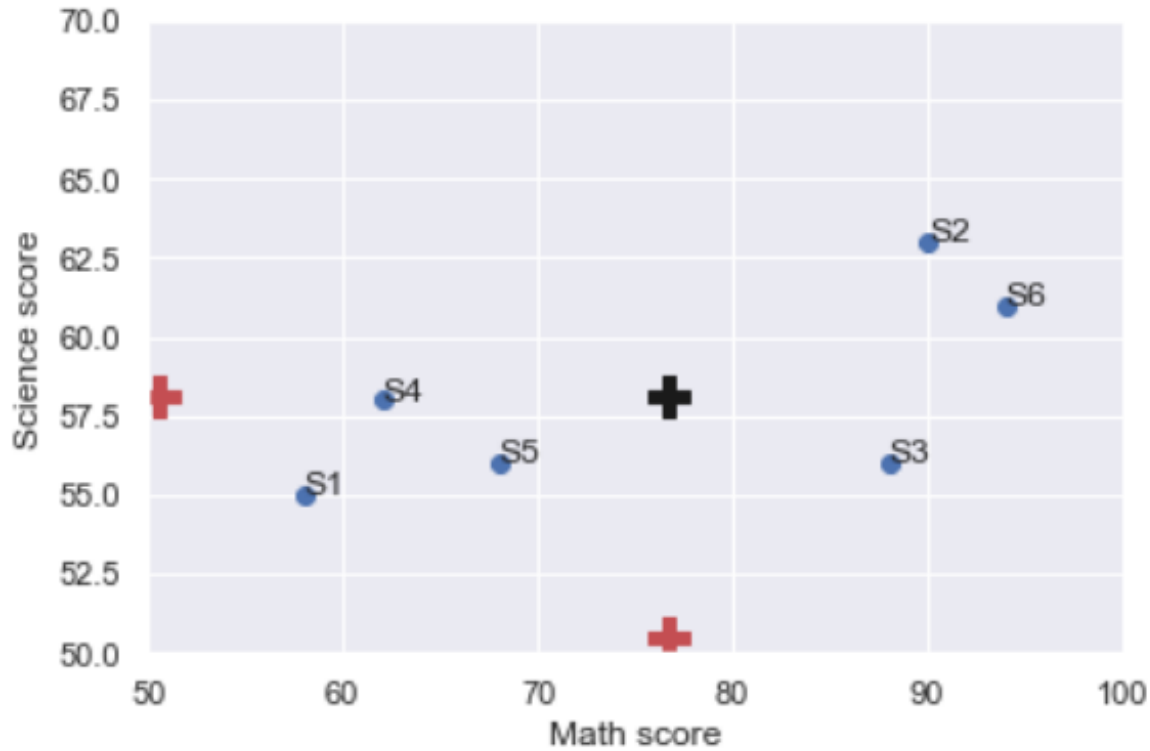
Recap: Eigenvectors and Eigen values

- Suppose $\mathbf{u}$ is a non-zero column vector with d components.
- We multiply $\mathbf{u}$ by a matrix $\mathbf{A}$: $\mathbf{A}\mathbf{u}$
- The result will be of a different length to that of original vector and will also be rotated.
- If the resultant vector lies in the same direction as $\mathbf{u}$, then no rotation happened during transformation but only scaling.
- In mathematical terms, if we represent the transformation with a matrix ($\mathbf{A}$), an eigenvector ($\mathbf{u}$) satisfies this equation: $\mathbf{A}\mathbf{u} = \lambda\mathbf{u}$.
- Here, λ is the eigenvalue, a number that tells us how much the eigenvector ($\mathbf{u}$) is scaled when ($\mathbf{A}$) is applied to it.

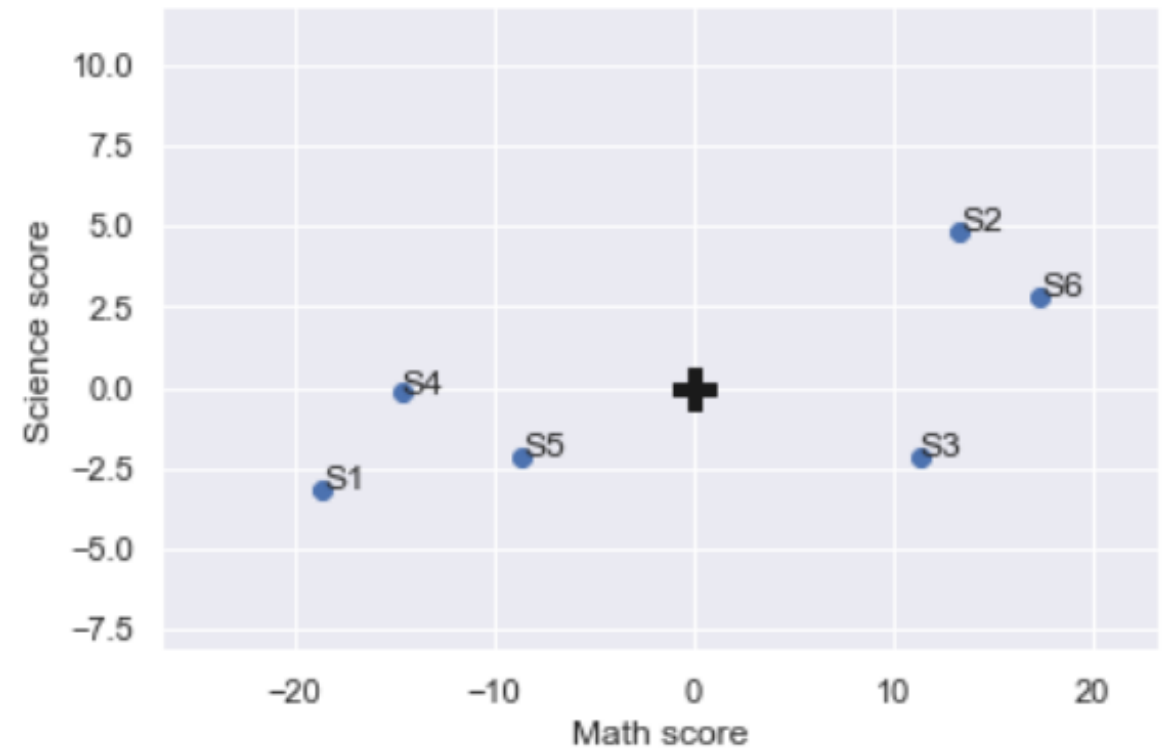
Principal Component Analysis (PCA)

- PCA is an extremely popular dimensionality reduction algorithm based on eliminating low-variance features.
- The first step in PCA is known as Karunen-Loève (KL) Transform, for feature decorrelation.
- The first d individual transformed features with largest variance is retained.
- Thus, PCA is **feature extraction algorithm** and not a **feature selection algorithm** owing to KL Transform.
- KL Transform being linear, interpretability is not lost after PCA.
- Further, the coefficients of the linear transform provide information about the relative importance of the original features.
- However, classical PCA is unsupervised.

PCA Step I: Center the dataset



$$\tilde{X} = (X - \mu_X)$$



PCA Step II: Covariance Matrix

- Compute the covariance matrix for the dataset:

$$\Sigma = \begin{bmatrix} \text{Var}(X_1) & \text{Cov}(X_1, X_2) \\ \text{Cov}(X_1, X_2) & \text{Var}(X_2) \end{bmatrix}$$

- Where,

$$\text{Var}(X) = \frac{\sum_{i=1}^n (x_i - \mu)^2}{n}$$
$$\text{Cov}(X_1, X_2) = \frac{\sum_{i=1}^n (x_{1,i} - \mu_{x_1})(x_{2,i} - \mu_{x_2})}{n}$$

- We get,

$$\Sigma = \begin{bmatrix} 249.07 & 33.87 \\ 33.87 & 10.17 \end{bmatrix}$$

PCA Step III: Eigen Values

- Let $A = \Sigma$, we need to find u satisfying:

$$Au = \lambda u$$

$Au = \lambda Iu$, where I is the identity matrix

$$(A - \lambda I)u = 0$$

- If the inverse of $(A - \lambda I)$ existed, we could multiply both sides of the equation by this inverse, resulting in $u = 0$ (the trivial solution).
- But by definition, eigenvectors cannot be zero vectors. Therefore, $(A - \lambda I)$ must be non-invertible for eigenvalues. Thus,

$$|A - \lambda I| = 0$$

PCA Step III: Eigen Values

$$\begin{aligned} |A - \lambda I| &= 0 \\ \lambda I &= \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix} \\ A - \lambda I &= \begin{bmatrix} 249.07 - \lambda & 33.87 \\ 33.87 & 10.17 - \lambda \end{bmatrix} \end{aligned}$$

- Calculating determinant,

$$\begin{aligned} |A - \lambda I| &= (249.07 - \lambda)(10.17 - \lambda) - 33.87 \times 33.87 = 0 \\ \lambda^2 - 259.24\lambda + 1385.87 &= 0 \end{aligned}$$

- Using quadratic formula,

$$\begin{aligned} \lambda &= \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \\ a &= 1, \quad b = -259.24, \quad c = 1385.87 \end{aligned}$$

- Thus,

$$\lambda_1 = 253.77, \quad \lambda_2 = 5.46$$

PCA Step IV: Eigenvectors

- We can find the eigenvectors using eigenvalues,

$$\begin{bmatrix} 249.07 & 33.87 \\ 33.87 & 10.17 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \lambda \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

- With $\lambda_1 = 253.77$,

$$\begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} 0.99 \\ 0.14 \end{bmatrix}$$

- With $\lambda_2 = 5.46$,

$$\begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} -0.14 \\ 0.99 \end{bmatrix}$$

- The vector, $\mathbf{u}$ with the highest eigen value (here $\lambda_1 = 253.77$) corresponds to first principal component PC1 and the remaining corresponds to PC2.
- To project the data points into a new space with PC1, PC2 as axis, we calculate:

$$\mathbf{Z} = \mathbf{U}^T (\mathbf{X} - \boldsymbol{\mu}_X)$$

where $\mathbf{U} = \begin{bmatrix} 0.99 & -0.14 \\ 0.14 & 0.99 \end{bmatrix}$

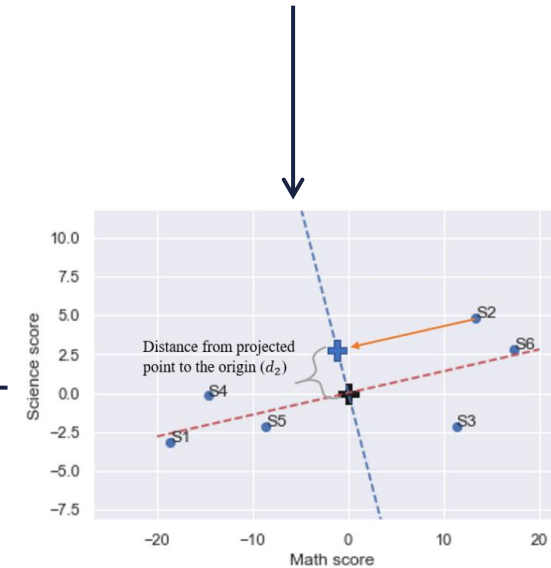
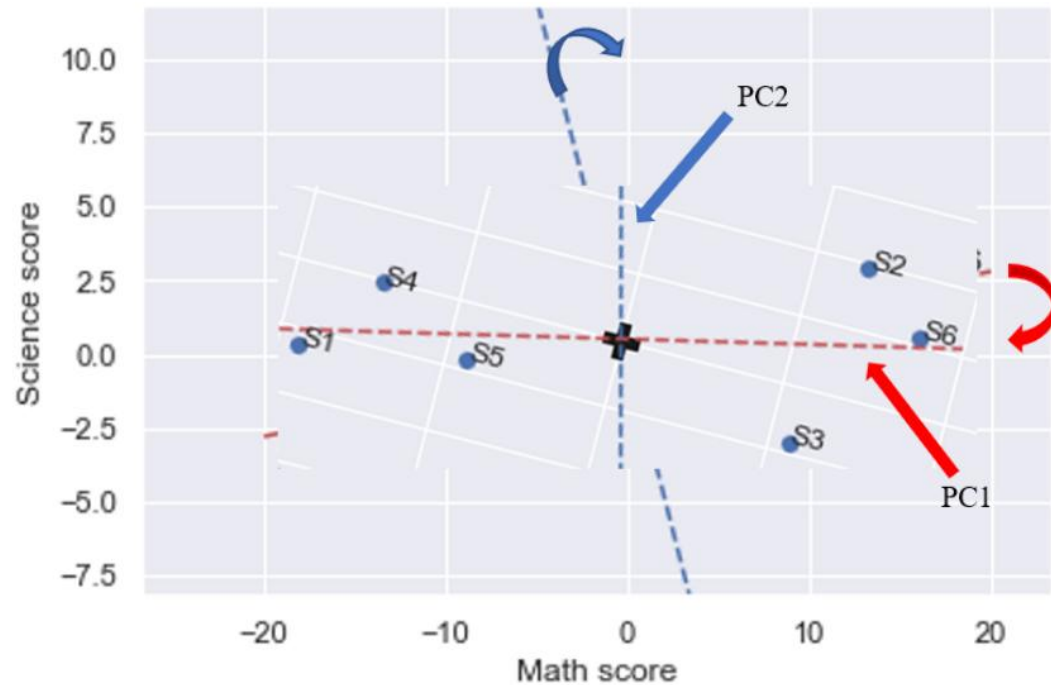
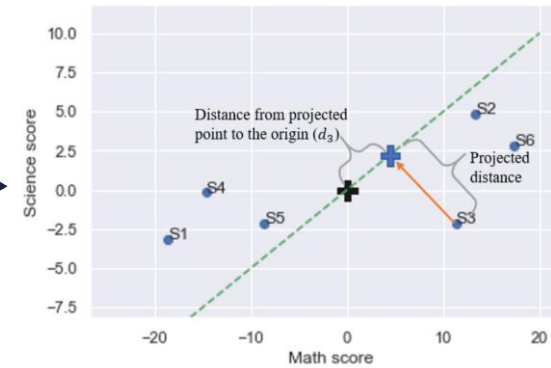
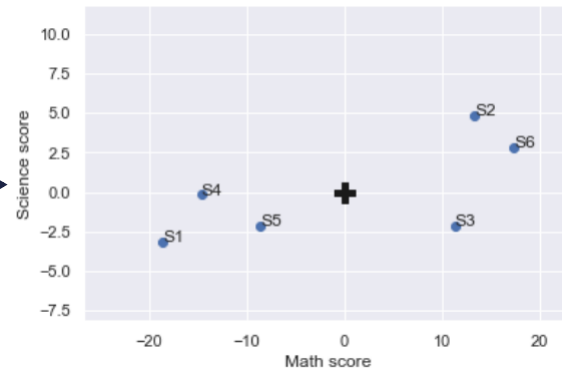
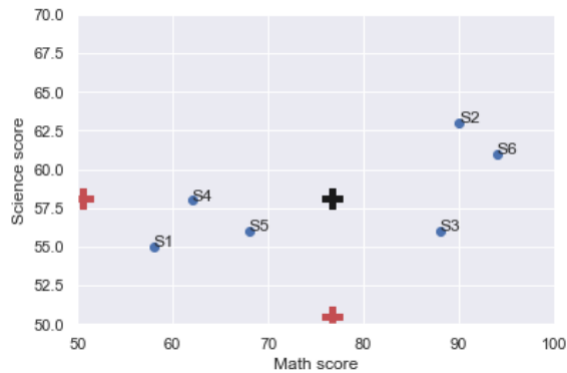
- The information content in the principal components $\mathbf{Z}$ can be calculated as:

$$Z_1 \rightarrow \frac{\lambda_1}{\lambda_1 + \lambda_2} \times 100 = 97.89\%$$

$$Z_2 \rightarrow \frac{\lambda_2}{\lambda_1 + \lambda_2} \times 100 = 2.11\%$$

PCA: Intuition

$$\lambda_1 = \sum_i d_i^2$$



$$\lambda_2 = \sum_i d_i^2$$

Principal Component Analysis (PCA)

- Consider the feature vector $\mathbf{X} \in \mathbb{R}^p$ with mean $\boldsymbol{\mu}_X$ and covariance matrix Σ_X .
- Σ_X has a set of p orthogonal eigenvectors $\mathbf{u}_1, \dots, \mathbf{u}_p$ with nonnegative eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$.
- Consider the linear transformation given by:

$$\mathbf{Z} = U^T(\mathbf{X} - \boldsymbol{\mu}_X)$$

where $U = [\mathbf{u}_1, \dots, \mathbf{u}_p]$ is the matrix of eigenvectors.

- This is a rotation in the feature-space known as discrete Karunen-Loève (KL) Transform, preceded by a translation to remove the mean and is
- Now,

$$E[\mathbf{Z}] = E[U^T(\mathbf{X} - \boldsymbol{\mu}_X)] = U^T(E[\mathbf{X}] - \boldsymbol{\mu}_X) = 0$$

Principal Component Analysis (PCA)

- And,

$$\begin{aligned}\Sigma_Z &= E[\mathbf{Z}\mathbf{Z}^T] = E[U^T(\mathbf{X} - \boldsymbol{\mu}_X)(\mathbf{X} - \boldsymbol{\mu}_X)^T U] \\ &= U^T E[(\mathbf{X} - \boldsymbol{\mu}_X)(\mathbf{X} - \boldsymbol{\mu}_X)^T] U = U^T \Sigma_X U = \Lambda\end{aligned}$$

Λ is the diagonal matrix of eigenvalues $\lambda_1, \dots, \lambda_p$

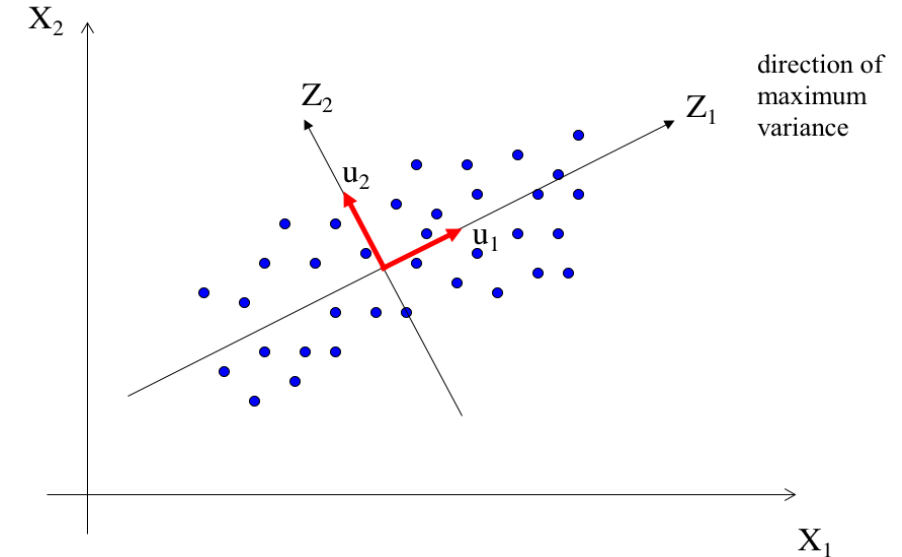
- With $\mathbf{Z} = (Z_1, \dots, Z_p)$, the aforementioned derivation proves that:

$$\begin{aligned}E[Z_i] &= 0, & \forall i = 1, \dots, p, \\ E[Z_i Z_j] &= 0, & \forall i, j = 1, \dots, p, i \neq j, \\ E[Z_i^2] &= \text{Var}(Z_i) = \lambda_i, & \forall i = 1, \dots, p.\end{aligned}$$

- Z_i is zero-mean, uncorrelated random variables, with variance equal to the eigenvalues of Σ_X .

Principal Component Analysis (PCA)

- Since $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$, $\text{Var}(Z_1) \geq \text{Var}(Z_2) \geq \dots \geq \text{Var}(Z_p) \geq 0$
- u_1 points to the direction of maximal variance. $Z_1, Z_2, Z_3 \dots$ are called the first, second, third, and so on principal components.
- u_2 points in the direction of largest variance perpendicular to u_1 , u_3 points to the largest variance perpendicular to the space spanned by u_1 , and u_2 , and so on.
- KL-transform illustrated with $p = 2$. The data is uncorrelated in the rotated axis system $u_1 \times u_2$.



Principal Component Analysis (PCA)

- The PCA transform $X' = T(X)$ for feature extraction consists of:
 - Centering the data
 - Applying KL-transform
 - Keeping the first d principal components $X' = (Z_1, \dots, Z_d)$
- Thus,

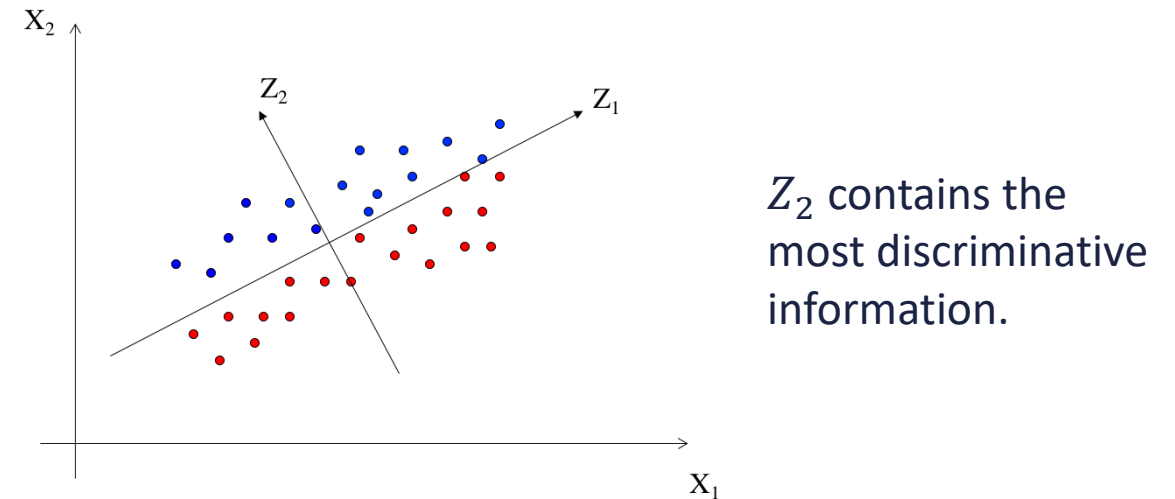
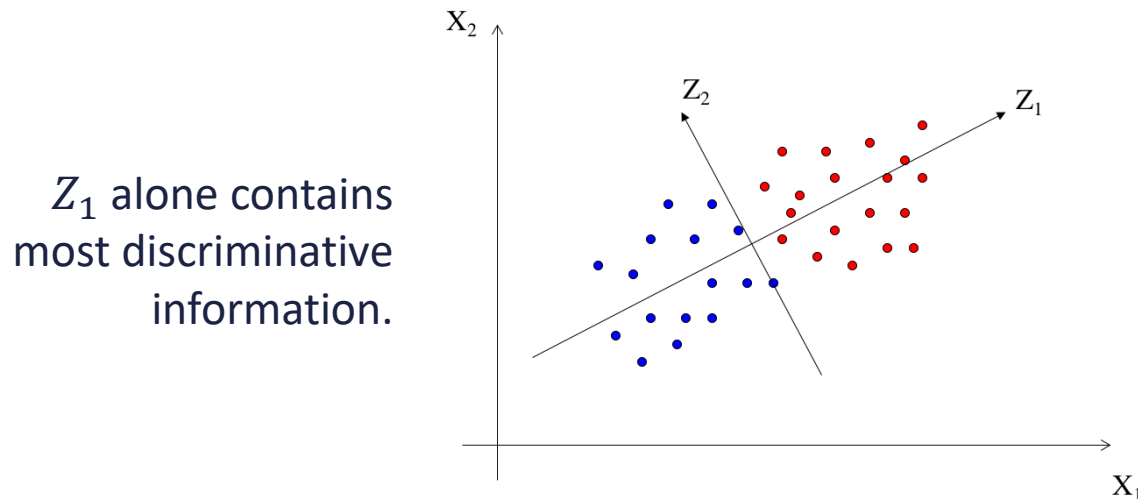
$$X' = U^T (X - \mu_X)$$

with $U = [\mathbf{u}_1, \dots, \mathbf{u}_d]$ called the **loading matrix**.

- PCA is generally not invertible and lossy.
- In practice we work with samples $S_n = \{X_1, \dots, X_n\}$, where the mean μ_X and covariance matrix Σ_X is calculated using the samples.

Principal Component Analysis (PCA)

- The issue with PCA is that it doesn't consider the labels.
- The assumption with PCA is that the discriminatory information is usually contained in the direction of largest variance.
- Suppose that, in this artificial example, it is desired to perform dimensionality reduction from $p = 2$ to $d = 1$ features.



Thank You